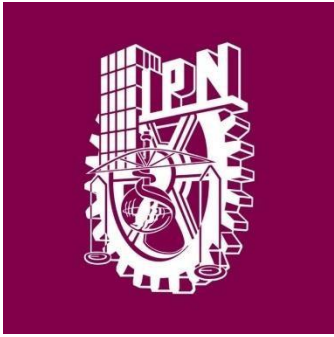




Instituto Politécnico
Nacional
Escuela Superior de
Computo



Profesor: Cortes Galicia Jorge

Materia: Sistemas Operativos

Practica 4

Integrantes:

Vázquez Blancas Cesar Said

Díaz Torres Jonathan Samuel

Aparicio Ponce Roberto Manuel

Corona Hernández Martín Rafael

Introducción Teórica

La programación en el intérprete de comandos, también conocida como scripting en shell, es una técnica utilizada para automatizar tareas y operaciones en sistemas operativos basados en Unix, como Linux, y en sistemas basados en DOS/Windows. Esta técnica permite la creación de secuencias de comandos que interactúan directamente con el núcleo del sistema operativo, facilitando la ejecución de acciones repetitivas o complejas de manera más eficiente y controlada.

El Intérprete de Comandos en Linux: Shell Bash

En los sistemas Linux, el intérprete de comandos más comúnmente utilizado es Bash (Bourne Again Shell), un sucesor mejorado del Bourne Shell original. Bash permite a los usuarios ejecutar comandos directamente desde una línea de comandos, pero también admite la creación de scripts, archivos que contienen una serie de instrucciones que el sistema puede ejecutar de forma secuencial. La programación en Bash es extremadamente potente debido a su capacidad para combinar comandos simples con estructuras de control, como bucles, condicionales y funciones, lo que permite realizar operaciones complejas con relativamente pocas líneas de código.

Características principales de Bash:

- **Interactividad:** Bash permite ejecutar comandos interactivos desde la línea de comandos, proporcionando un entorno donde el usuario puede escribir comandos, obtener resultados, y ejecutar scripts.
- **Automatización:** Los scripts de Bash se pueden utilizar para automatizar tareas repetitivas, como copias de seguridad, administración de usuarios, gestión de archivos y mantenimiento del sistema.
- **Integración con herramientas Unix:** Bash interactúa de manera eficiente con las herramientas básicas de Unix/Linux, como grep, awk, sed, find, y tar, entre otros. Esto facilita la manipulación de texto, archivos y procesos del sistema.
- **Soporte para variables y funciones:** Bash permite declarar variables, funciones y utilizar argumentos de entrada, lo que aumenta la flexibilidad y reutilización del código.
- **Control de flujo:** Al igual que en otros lenguajes de programación, Bash incluye estructuras de control de flujo, como condicionales (if, else, elif), bucles (for, while, until), y mecanismos para manejar errores y excepciones.

El Intérprete de Comandos en Windows: Batch Scripting

En los sistemas Windows, el Batch scripting (o programación de archivos por lotes) utiliza el intérprete de comandos CMD.exe o Command Prompt. Los scripts por lotes

son archivos con extensión .bat que contienen una lista de comandos a ejecutarse de manera secuencial. Al igual que en Bash, los scripts batch se utilizan para automatizar tareas repetitivas, aunque las capacidades del scripting batch son más limitadas en comparación con Bash.

Características principales de Batch:

- Simplicidad: Batch scripting ofrece una forma simple de automatizar tareas básicas en Windows, como la creación, modificación y eliminación de archivos y directorios.
- Interfaz de línea de comandos: Los scripts por lotes se ejecutan a través de la interfaz de línea de comandos de Windows, CMD.exe, lo que permite interactuar directamente con el sistema de archivos y otros componentes del sistema operativo.
- Compatibilidad heredada: Batch scripting ha sido parte de todos los sistemas operativos Windows desde los días de MS-DOS, lo que garantiza una amplia compatibilidad con versiones antiguas de Windows.
- Comandos básicos: Los scripts batch permiten el uso de comandos simples como echo (para mostrar mensajes en pantalla), del (para eliminar archivos), y date (para mostrar o modificar la fecha del sistema).

Comparativa entre Bash y Batch

Aunque tanto Bash como Batch scripting cumplen funciones similares en sus respectivos entornos, existen algunas diferencias clave en cuanto a su funcionalidad y uso:

1. Versatilidad y poder: Bash es mucho más versátil y poderoso que el scripting por lotes de Windows, ya que proporciona un entorno de programación más completo con soporte para funciones avanzadas, manejo de procesos, y manipulación de archivos. Además, se integra con una vasta cantidad de herramientas Unix que permiten la creación de scripts complejos.
2. Facilidad de uso: Batch scripting es más fácil de aprender para principiantes, ya que tiene una sintaxis más simple y está enfocado en tareas básicas. Sin embargo, su capacidad es limitada cuando se trata de operaciones más avanzadas.
3. Compatibilidad entre sistemas: Los scripts en Bash solo pueden ejecutarse en sistemas Unix/Linux, aunque hay herramientas como Cygwin que permiten ejecutar Bash en Windows. Por otro lado, los scripts batch son específicos de Windows y no pueden ejecutarse en Linux sin algún tipo de emulación.

Sección Linux:

1. **A través de la ayuda en línea que proporciona Linux, investigue para que es utilizado el comando bash, mencione sus características principales, dé un resumen de su funcionalidad.**

El comando bash (Bourne Again Shell) es un intérprete de comandos en sistemas Unix y Linux, y es la shell predeterminada en la mayoría de las distribuciones de Linux. Se utiliza para ejecutar comandos, scripts y programas, interactuando directamente con el sistema operativo.

Características principales de Bash:

1. **Interactividad:** Bash permite a los usuarios interactuar directamente con el sistema a través de la línea de comandos, ingresando y ejecutando comandos.
2. **Automatización:** Mediante la creación de scripts, los usuarios pueden automatizar tareas repetitivas, ejecutando múltiples comandos de manera secuencial o condicional.
3. **Variables:** Bash soporta variables para almacenar y manipular datos. Existen variables de entorno que el sistema utiliza para el funcionamiento general (como PATH, HOME) y variables definidas por el usuario.
4. **Funciones:** Permite la definición de funciones dentro de scripts para reutilizar bloques de código, lo que facilita la organización y legibilidad del script.
5. **Control de flujo:** Incluye estructuras de control como condicionales (if, else, elif), bucles (for, while), y declaraciones de caso (case), que permiten la toma de decisiones dentro de los scripts.
6. **Gestión de archivos y procesos:** Bash ofrece comandos integrados para gestionar archivos, carpetas y procesos, como cd, ls, cp, mv, ps, entre otros.
7. **Redirección de E/S:** Bash soporta redirección de entrada y salida estándar, lo que permite redirigir el flujo de datos entre comandos y archivos mediante operadores como >, >>, <, |.

Funcionalidad de Bash:

Bash actúa como un intermediario entre el usuario y el núcleo (kernel) del sistema operativo, ejecutando los comandos que se le proporcionan. Los usuarios pueden escribir comandos de forma interactiva o utilizar scripts Bash para ejecutar múltiples tareas automáticamente. Bash también es capaz de interpretar scripts escritos en otros lenguajes, como **Python** o **Perl**, siempre que estén correctamente configurados en el sistema.

En resumen, **Bash** es una herramienta poderosa para interactuar con el sistema operativo, permitiendo tanto la ejecución de tareas básicas de administración como la automatización avanzada mediante scripts.

2. Revise en la ayuda en línea de Linux los siguientes comandos: **date**, **tar**, **bzip2**, **bunzip2**. Dé un resumen de sus características y funcionalidad.

1. Comando **date**:

- Función: Muestra la fecha y hora actuales del sistema, o permite establecerlas.
- Características:
 - Permite formatear la salida de la fecha según distintas especificaciones (día, mes, año, hora, etc.).
 - Se puede usar para mostrar la fecha en un formato personalizado utilizando opciones como %Y (año), %m (mes), %d (día), %H (hora), etc.
 - Puede mostrar la fecha y hora actual o ajustarse para mostrar fechas futuras o pasadas utilizando operadores aritméticos.
 - Ejemplo de uso:
 - Mostrar la fecha actual: `date`
 - Mostrar la fecha en un formato específico: `date +"%Y-%m-%d %H:%M:%S"`

2. Comando **tar**:

- Función: Se utiliza para archivar varios archivos en un solo archivo comprimido o sin comprimir.
- Características:
 - Combina varios archivos y directorios en un solo archivo llamado tarball.
 - Los archivos generados suelen tener la extensión .tar.
 - Puede utilizarse en conjunto con otros programas de compresión (como gzip o bzip2) para generar archivos comprimidos .tar.gz o .tar.bz2.
 - Permite crear (-c), extraer (-x), y listar (-t) archivos de un tarball.

- Ejemplo de uso:
 - Crear un archivo tar: `tar -cvf archivo.tar directorio/`
 - Extraer un archivo tar: `tar -xvf archivo.tar`

3. Comando bzip2:

- Función: Se utiliza para comprimir archivos utilizando el algoritmo de compresión Burrows-Wheeler.
- Características:
 - Comprime archivos individuales, reemplazando el archivo original por su versión comprimida.
 - Los archivos comprimidos con bzip2 tienen la extensión `.bz2`.
 - Es más lento que gzip, pero generalmente ofrece mejores tasas de compresión.
 - Solo puede comprimir archivos individuales, por lo que suele utilizarse junto con tar para comprimir un conjunto de archivos.
 - Ejemplo de uso:
 - Comprimir un archivo: `bzip2 archivo.txt` (esto generará `archivo.txt.bz2`)

4. Comando bunzip2:

- Función: Se utiliza para descomprimir archivos que han sido comprimidos con bzip2.
- Características:
 - Descomprime archivos `.bz2`, restaurando el archivo original.
 - Se ejecuta automáticamente sobre archivos comprimidos con bzip2.
 - Similar al comando bzip2, pero en lugar de comprimir, descomprime.
 - Ejemplo de uso:
 - Descomprimir un archivo: `bunzip2 archivo.txt.bz2` (esto generará `archivo.txt`)

3.- Capture y ejecute el siguiente script. Observe su funcionamiento y explique.

```
#!/bin/sh
#
clear
#
ls -l
#
rm -f ./prueba
#
ls -l
#
echo "----- Creando archivo: prueba -----"
#
echo "Hola a todos" >> prueba
echo "Esto es una prueba" >> prueba
echo "De un script" >> prueba
#
ls -l ./prueba
#
echo "----- Script terminado -----"
more ./prueba
```

Para ejecutar el script cambie los permisos de su archivo para permitir ejecución a nivel dueño. Ejecute como un programa normal en C.

```
total 4
-rwxrwxr-x 1 said said 317 oct 11 20:21 script.sh
total 4
-rwxrwxr-x 1 said said 317 oct 11 20:21 script.sh
----- Creando archivo: prueba -----
-rw-rw-r-- 1 said said 45 oct 11 20:24 ./prueba
----- Script terminado -----
Hola a todos
Esto es una prueba
De un script
```

El script inicia con `#!/bin/sh` para ser ejecutado por el shell, seguido de `clear` para limpiar la pantalla. Luego, ejecuta `ls -l` para listar los archivos en el directorio actual y usa `rm -f ./prueba` para eliminar el archivo prueba si existe. Después, ejecuta otro `ls -l` para mostrar los archivos tras la eliminación. A continuación, crea el archivo prueba y añade tres líneas de texto con `echo`. Tras esto, lista los detalles del nuevo archivo con `ls -l ./prueba` y finalmente usa `more ./prueba` para mostrar su contenido en pantalla.

4. Capture y ejecute el siguiente script. Sustituya las rutas utilizadas en el script por las rutas que usted utilice. Observe su funcionamiento y explique.

```
#!/bin/sh
#
clear
#
echo "----- Obteniendo fecha -----"
fecha=`date +%Y%m%d%H%M`
#
echo "----- Empacando -----"
tar -cvf ./respaldo$fecha.tar /home/said/directorio1 /home/said/directorio2 /home/directorio3
#
echo "----- Comprimiendo -----"
bzip2 ./respaldo$fecha.tar
#
echo "----- Enviando a otro directorio -----"
cp ./respaldo$fecha.tar.bz2 /home/said/respaldos
#
echo "----- Descomprimiendo -----"
bunzip2 /home/said/respaldos/respaldo$fecha.tar.bz2
#
echo "----- Desempacando -----"
tar -xvf /home/said/respaldos/respaldo$fecha.tar
#
echo "----- Limpiando -----"
rm -f ./respaldo$fecha.tar.bz2
rm -f ./respaldo$fecha.tar
#
echo "----- Terminado -----"
```



```

----- Obteniendo fecha -----
----- Empacando -----
tar: Eliminando la '/' inicial de los nombres
/home/said/directorio1/
/home/said/directorio1/directorio1.txt
tar: Eliminando la '/' inicial de los objetivos de los enlaces
/home/said/directorio2/
/home/said/directorio2/directorio2.txt
/home/directorio3/
/home/directorio3/directorio3.txt
----- Comprimiendo -----
----- Enviando a otro directorio -----
----- Descomprimiendo -----
----- Desempacando -----
home/said/directorio1/
home/said/directorio1/directorio1.txt
home/said/directorio2/
home/said/directorio2/directorio2.txt
home/directorio3/
home/directorio3/directorio3.txt
----- Limpiando -----
----- Terminado -----

```

```

said@said-VirtualBox:~/Descargas/home$ ls -R
.:
directorio3  said

./directorio3:
directorio3.txt

./said:
directorio1  directorio2

./said/directorio1:
directorio1.txt

./said/directorio2:
directorio2.txt

```



respaldo202
410112207.t
ar

El script comienza limpiando la pantalla con clear y luego obtiene la fecha y hora actual en formato YYYYMMDDHHMM, asignándola a la variable fecha. A

continuación, usa el comando tar para crear un archivo comprimido llamado respaldo\$fecha.tar, que contiene los directorios. Luego, comprime este archivo con bzip2, generando respaldo\$fecha.tar.bz2, y lo copia al directorio de respaldos. Después, descomprime el archivo con bunzip2 y lo desempaca con tar -xvf. Finalmente, limpia los archivos temporales eliminando el archivo .tar.bz2 y el .tar, concluyendo con un mensaje de finalización

5. Capture y ejecute el siguiente script. Observe su funcionamiento.

```
#!/bin/sh
clear
echo -n "Introduce dos valores numéricos separados por espacios: "
read var1 var2
echo "Los valores son:"
echo "var1=$var1"
echo "var2=$var2"
if [ $var1 -lt $var2 ]; then
    echo "$var1 es menor que $var2"
    while [ $var1 -lt $var2 ]; do
        echo "contador=$var1"
        var1=`expr $var1 + 1`
    done
else
    echo "$var1 es mayor o igual que $var2"
    case $var2 in
        10) echo "Fui un 10";;
        20) echo "Fui un 20";;
        *) echo "Otro valor";;
    esac
fi
```

```
Introduce dos valores numéricos separados por espacios: 18 26
Los valores son:
var1=18
var2=26
18 es menor que 26
contador=18
contador=19
contador=20
contador=21
contador=22
contador=23
contador=24
contador=25
```

```
Introduce dos valores numéricos separados por espacios: 10 7
Los valores son:
var1=10
var2=7
10 es mayor o igual que 7
Otro valor
```

```
Introduce dos valores numéricos separados por espacios: 15 10
Los valores son:
var1=15
var2=10
15 es mayor o igual que 10
Fui un 10
```

Este script comienza limpiando la pantalla con el comando clear, luego solicita al usuario que introduzca dos valores numéricos separados por espacios con el mensaje Introduce dos valores numéricos separados por espacios. Los valores ingresados se almacenan en las variables var1 y var2 usando el comando read. Después, imprime los valores ingresados con echo para mostrar var1 y var2. A continuación, el script evalúa si var1 es menor que var2 utilizando un bloque condicional if. Si esta condición es verdadera, imprime que var1 es menor y entra en un bucle while que incrementa var1 en uno en cada iteración hasta que sea igual o mayor que var2, mostrando el valor de var1 en cada paso. Si var1 es mayor o igual que var2, el script imprime este hecho y usa una estructura case para evaluar el valor de var2. Dependiendo de si var2 es 10, 20, u otro valor, muestra un mensaje correspondiente. Finalmente, el script termina después de ejecutar el bloque condicional o el bucle, dependiendo de los valores ingresados.

6. Capture y ejecute el siguiente script. Observe su funcionamiento.

```
#!/bin/bash
clear
Op="Opcion: "
echo $Op
select i in Directorio DirectorioExtendido Salir
do
    case $i in
        Directorio)
            ls
            ;;
        DirectorioExtendido)
            ls -l
            ;;
        Salir)
            break
            ;;
        *)
            echo "Opción inválida"
            ;;
    esac
done
echo ""
for i in hola como estan todos
do
    echo -n "$i "
done
echo ""
```

```
Opcion:
1) Directorio
2) DirectorioExtendido
3) Salir
#? 7
Opción inválida
#? 1
home prueba script2.sh script3.sh script4.sh script.sh
#? 2
total 24
drwxrwxr-x 4 said said 4096 oct 11 22:07 home
-rw-rw-r-- 1 said said  45 oct 11 22:21 prueba
-rwxrwxr-x 1 said said  819 oct 11 21:58 script2.sh
-rwxrwxr-x 1 said said  501 oct 11 22:21 script3.sh
-rwxrwxr-x 1 said said  434 oct 11 22:30 script4.sh
-rwxrwxr-x 1 said said  317 oct 11 20:21 script.sh
#? 4
Opción inválida
#? 3

hola como estan todos
```

El script comienza limpiando la pantalla con el comando clear y luego define una variable Op con el texto "Opción: ". A continuación, utiliza el comando select para presentar un menú con tres opciones: "Directorio", "DirectorioExtendido" y "Salir". Dependiendo de la opción seleccionada por el usuario, se ejecuta un bloque case. Si se elige "Directorio", el comando ls lista los archivos y directorios en el directorio actual. Si se selecciona "DirectorioExtendido", el comando ls -l proporciona una lista detallada de los archivos, incluyendo permisos, tamaño y fecha de modificación. Si el usuario elige "Salir", el script se detiene usando break. Después de que se sale del menú, el script entra en un bucle for que itera sobre las palabras "hola", "como", "estan" y "todos", imprimiendo cada una de ellas en la misma línea, sin salto de línea entre ellas debido a la opción -n en el comando echo. Finalmente, el script termina tras ejecutar las instrucciones del menú y del bucle.

7. Programe un script para que descomprima en sus rutas originales los directorios que se han compreso con el anterior script

```
#!/bin/bash
clear
directorio_respaldo="/home/said/respaldos"
if [ ! -d "$directorio_respaldo" ]; then
    echo "El directorio $directorio_respaldo no existe."
    exit 1
fi
for archivo in "$directorio_respaldo"/*.tar; do
    if [ ! -e "$archivo" ]; then
        echo "No se encontraron archivos .tar en $directorio_respaldo."
        exit 1
    fi
    echo "Descomprimiendo $archivo..."
    ruta_destino="/home/said/Descomprimidos"
    mkdir -p "$ruta_destino"
    tar -xvf "$archivo" -C "$ruta_destino"
    echo "Descompresión completada para $archivo"
done
echo "Todas las descompresiones han finalizado."
```

```
Descomprimiendo /home/said/respaldos/respaldo202410112207.tar...
home/said/directorio1/
home/said/directorio1/directorio1.txt
home/said/directorio2/
home/said/directorio2/directorio2.txt
home/directorio3/
home/directorio3/directorio3.txt
Descompresión completada para /home/said/respaldos/respaldo202410112207.tar
Todas las descompresiones han finalizado.
```

```
said@said-VirtualBox:~$ cd Descomprimidos
said@said-VirtualBox:~/Descomprimidos$ ls -R
.:
home

./home:
directorio3  said

./home/directorio3:
directorio3.txt

./home/said:
directorio1  directorio2

./home/said/directorio1:
directorio1.txt

./home/said/directorio2:
directorio2.txt
```

< > Carpeta personal / respaldos



respaldo202
410112207.t
ar

8. Programe un script generador de scripts, es decir, que cree un archivo, que posteriormente permita editarlo y que finalmente cambie los permisos para ejecución del archivo, mandando a ejecutar el script desde el script original.

```
#!/bin/bash
g="generado.sh"
echo "Creando el archivo $g..."
cat <<EOL > $g
#!/bin/bash
echo "Este es un script generado por otro script"
EOL
echo "Editando el archivo $g..."
nano $g
echo "Cambiando permisos del archivo $g..."
chmod +x $g
echo "Ejecutando el archivo $g..."
./$g
```

```
Creando el archivo generado.sh...
Editando el archivo generado.sh...
Cambiando permisos del archivo generado.sh...
Ejecutando el archivo generado.sh...
Este es un script generado por otro script
adios desde nano
```



```
GNU nano 7.2                                generado.sh
#!/bin/bash
echo "Este es un script generado por otro script"
```

[2 líneas leídas]

^G Ayuda	^O Guardar	^W Buscar	^K Cortar	^T Ejecutar	^C Ubicación
^X Salir	^R Leer fich.	^\\ Reemplazar	^U Pegar	^J Justificar	^/ Ir a línea

said@said-VirtualBox: ~/Descargas

```
GNU nano 7.2                                generado.sh *
```

```
#!/bin/bash
echo "Este es un script generado por otro script"
echo "adios desde nano"
```

^G Ayuda	^O Guardar	^W Buscar	^K Cortar	^T Ejecutar	^C Ubicación
^X Salir	^R Leer fich.	^\\ Reemplazar	^U Pegar	^J Justificar	^/ Ir a línea

9. Programe un script a través de las estructuras de control de flujo para que permita visualizar los archivos de un directorio que se envíe como argumento de entrada (no utilizar el comando ls). Además, el script deberá mostrar el número total de archivos en el directorio.

```
#!/bin/bash
if [ -z "$1" ]; then
    echo "Error"
    echo "Uso: $0 <directorio>"
    exit 1
fi
if [ ! -d "$1" ]; then
    echo "Error: $1 no es un directorio válido."
    exit 1
fi
contador=0
echo "Archivos en el directorio $1:"
for archivo in "$1"/*; do
    if [ -f "$archivo" ]; then
        echo "$(basename "$archivo")"
        contador=$((contador + 1))
    fi
done
echo "Número total de archivos: $contador"
```

```

said@said-VirtualBox:~/Descargas$ ./directorio.sh
Error
Uso: ./directorio.sh <directorio>
said@said-VirtualBox:~/Descargas$ ./directorio.sh /home/said/Documentos
Archivos en el directorio /home/said/Documentos:
balan
balancear.c
exp
expresion.c
figura
figura.c
hanoi
hanoi.c
holaMundo
holaMundo.c
Número total de archivos: 10
said@said-VirtualBox:~/Descargas$ cd ..
said@said-VirtualBox:~$ cd Documentos
said@said-VirtualBox:~/Documentos$ ls
balan      exp      figura   hanoi    holaMundo
balancear.c expresion.c figura.c hanoi.c  holaMundo.c
said@said-VirtualBox:~/Documentos$

```

10. Programe un script que elimine todos los archivos de un directorio especificado desde el propio script, y cuyo segundo carácter sea la letra “e”.

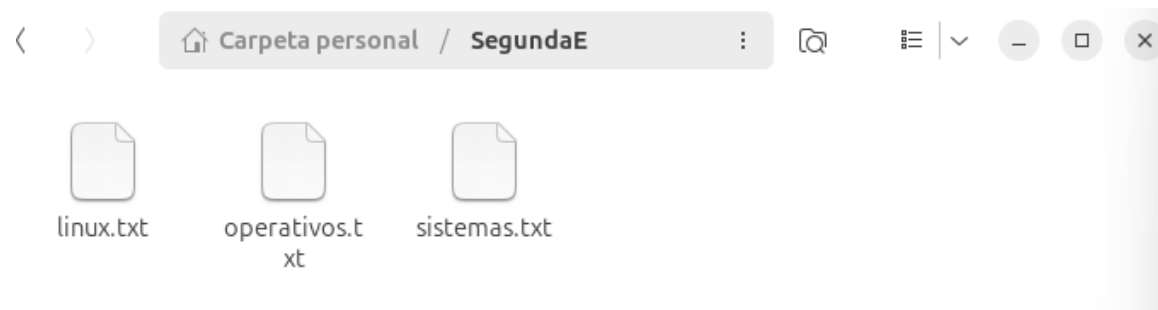
```

#!/bin/bash
directorio="/home/said/SegundaE"
if [ ! -d "$directorio" ]; then
    echo "Error: $directorio no es un directorio válido."
    exit 1
fi
contador=0
for archivo in "$directorio"/*; do
    if [ -f "$archivo" ] && [ "$(basename "$archivo")" =~ ^.e ]; then
        echo "Eliminando archivo: $(basename "$archivo")"
        rm "$archivo"
        contador=$((contador + 1))
    fi
done
echo "Total de archivos eliminados: $contador"

```



```
said@said-VirtualBox:~/Descargas$ ./eliminar.sh
Eliminando archivo: cero.txt
Eliminando archivo: fecha.txt
Eliminando archivo: tecla.txt
Total de archivos eliminados: 3
said@said-VirtualBox:~/Descargas$
```



Sección Windows:

1. Investigue los siguientes comandos: echo, del, date, tar, rar. Dé un resumen de sus características y funcionalidad.

1. echo

- Descripción: El comando echo en Windows muestra mensajes o el contenido de variables en la línea de comandos.
- Funcionalidad:
 - Se utiliza para imprimir texto en la consola o scripts por lotes (batch files).
 - Muestra el valor de variables de entorno.
 - Puede controlar si los comandos se muestran o no en la consola en un script de lote (usando @echo off para ocultar los comandos).
- Ejemplos:

echo Hola Mundo # Imprime "Hola Mundo" en la consola.

echo %USERNAME% # Muestra el nombre del usuario actual (valor de la variable USERNAME).

@echo off # Desactiva la visualización de comandos en un script batch.

2. del

- Descripción: El comando del se utiliza para eliminar archivos en el sistema Windows.
- Funcionalidad:
 - Borra uno o varios archivos en un directorio.
 - No elimina directorios, solo archivos.
 - Se puede usar con comodines (*) para eliminar múltiples archivos a la vez.
 - Se puede forzar la eliminación sin confirmación con el modificador /F.
- Ejemplos:

del archivo.txt # Elimina el archivo "archivo.txt" en el directorio actual.

del C:\ruta\archivo.txt # Elimina un archivo especificado en una ruta.

del *.txt # Elimina todos los archivos con extensión .txt en el directorio actual.

del /F /Q archivo.txt # Fuerza la eliminación de "archivo.txt" sin confirmación (silenciosa).

3. date

- Descripción: El comando date en Windows muestra o establece la fecha del sistema.
- Funcionalidad:
 - Muestra la fecha actual del sistema.
 - Permite cambiar la fecha del sistema.
 - En caso de querer modificar la fecha, solicita ingresar la nueva en el formato adecuado (día/mes/año en muchos sistemas locales).
 - Si no se quiere cambiar la fecha y solo se desea ver, se puede presionar Enter al ser solicitado.
- Ejemplos:

date # Muestra la fecha actual del sistema y pide ingresar una nueva fecha.

date /T # Muestra solo la fecha actual sin posibilidad de cambiarla.

4. tar

- Descripción: En versiones recientes de Windows 10/11 (con Windows Subsystem for Linux o con PowerShell), se incluye el comando tar para empaquetar y extraer archivos tar, una característica común en Unix/Linux pero también soportada en Windows.
- Funcionalidad:
 - Crea archivos empaquetados .tar o .tar.gz (comprimidos).
 - Extrae archivos tar.
 - Útil para trabajar con archivos tar en Windows, especialmente cuando se manejan proyectos multiplataforma.
- Ejemplos:

tar -cvf archivo.tar archivo.txt # Crea un archivo tar con el archivo "archivo.txt".

tar -xvf archivo.tar # Extrae el contenido del archivo tar.

tar -czvf archivo.tar.gz carpeta/ # Crea y comprime una carpeta en un archivo tar.gz.

5. rar

- Descripción: El comando rar no viene por defecto en Windows, pero se puede instalar usando el software WinRAR o herramientas similares. Este comando permite comprimir y extraer archivos en formato .rar.
- Funcionalidad:
 - Comprime archivos en un archivo .rar.
 - Extrae archivos comprimidos en .rar.

- Soporta compresión en múltiples volúmenes y archivos protegidos por contraseña.
- Se utiliza mayormente en la línea de comandos de WinRAR.
- Ejemplos:

`rar a archivo.rar archivo.txt` # Comprime el archivo "archivo.txt" en un archivo rar.

`rar x archivo.rar` # Extrae el contenido del archivo rar.

2. Capture y ejecute el siguiente batch. Guarde el archivo con extensión .bat. Observe su funcionamiento y explique.

```
@echo off
rem
cls
rem
dir .\prueba
rem
del .\prueba
rem
dir .\prueba
rem
echo "----- Creando archivo: prueba -----"
rem
echo "Hola a todos" >> prueba
echo "Esto es una prueba" >> prueba
echo "De un batch" >> prueba
rem
dir .\prueba
rem
echo "----- Script terminado -----"
type .\prueba
```

Para ejecutar escriba desde la línea de comando
>nombre_archivo.bat

```
El volumen de la unidad C es Windows-SSD
El número de serie del volumen es: C6A9-0150
```

```
Directorio de C:\Users\vbcs1
```

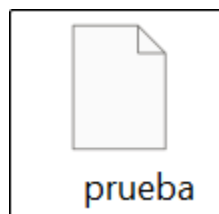
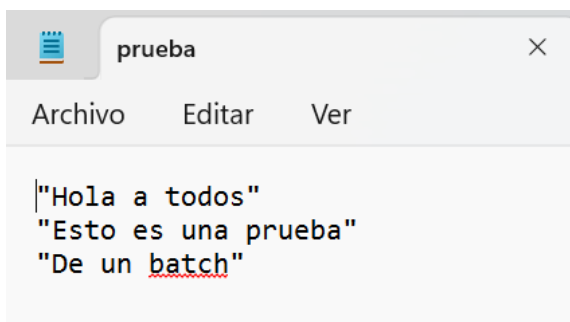
```
No se encuentra el archivo
No se pudo encontrar C:\Users\vbcs1\prueba
El volumen de la unidad C es Windows-SSD
El número de serie del volumen es: C6A9-0150
```

```
Directorio de C:\Users\vbcs1
```

```
No se encuentra el archivo
"----- Creando archivo: prueba -----"
El volumen de la unidad C es Windows-SSD
El número de serie del volumen es: C6A9-0150
```

```
Directorio de C:\Users\vbcs1
```

```
12/10/2024  20:52                56 prueba
                1 archivos                56 bytes
                0 dirs 346.219.540.480 bytes libres
"----- Script terminado -----"
"Hola a todos"
"Esto es una prueba"
"De un batch"
```



Este script de batch comienza con `@echo off`, que desactiva la visualización de los comandos que se ejecutan, para una salida más limpia. Luego, el comando `cls` se utiliza para limpiar la pantalla del terminal. A continuación, el script ejecuta `dir .\prueba`, que lista el contenido del archivo prueba en el directorio actual, aunque es probable que no exista en este punto. Luego, usa `del .\prueba` para eliminar el archivo prueba, si existe, y ejecuta nuevamente `dir .\prueba` para mostrar que el archivo ha sido eliminado. Después, imprime un mensaje que indica la creación del archivo prueba. A continuación, se crean tres líneas de texto que se añaden al archivo prueba mediante redirección con `>>`: primero, se agrega "Hola a todos",

seguido de "Esto es una prueba" y, por último, "De un batch". Luego, el script ejecuta dir .\prueba para mostrar que el archivo se ha creado exitosamente. Finalmente, imprime un mensaje de finalización y utiliza el comando type .\prueba para mostrar el contenido del archivo prueba, que ahora incluye las tres líneas de texto añadidas anteriormente. El script concluye tras mostrar el contenido del archivo.

3. Capture y ejecute el siguiente batch. Sustituya las rutas utilizadas en el batch por las rutas que usted utilice. Deberá tener instalado el programa Winrar para utilizar el comando rar en el script. Observe su funcionamiento y explique.

```
@echo off
cls
echo "----- Captura fecha -----"
for /F "tokens=1,2,3 delims=/ " %%V in ('date /t') do set dia=%%V%%W%%X
set tiempo=%time%
for /F "tokens=1,2,3 delims=:" %%V in ("%tiempo%") do set tiempo=%%V%%W%%X
set fecha=%dia%%tiempo%
echo "----- Empacando -----"
tar -cvf ".\respaldo%fecha%.tar" "C:\Users\vbcs1\directorio1" "C:\Users\vbcs1\directorio2" "C:\Users\vbcs1\directorio3"
echo "----- Comprimiendo -----"
"C:\Program Files\WinRAR\rar.exe" a -c- ".\respaldo%fecha%.rar" ".\respaldo%fecha%.tar"
echo "----- Enviando a otro directorio -----"
copy ".\respaldo%fecha%.rar" "C:\Users\vbcs1\Respaldos"
echo "----- Descomprimiendo -----"
"C:\Program Files\WinRAR\rar.exe" e -y "C:\Users\vbcs1\Respaldos\respaldo%fecha%.rar" "C:\Users\vbcs1\Respaldos"
echo "----- Desempacando -----"
tar -xvf "C:\Users\vbcs1\Respaldos\respaldo%fecha%.tar" -C "C:\Users\vbcs1\Respaldos"
echo "----- Limpiando -----"
del ".\respaldo%fecha%.rar"
del ".\respaldo%fecha%.tar"
echo "----- Terminado -----"
```

```
"----- Captura fecha -----"
"----- Empacando -----"
tar: Removing leading drive letter from member names
a Users/vbcs1/directorio1
a Users/vbcs1/directorio1/directorio1
a Users/vbcs1/directorio2
a Users/vbcs1/directorio2/directorio2
a Users/vbcs1/directorio3
a Users/vbcs1/directorio3/directorio3
"----- Comprimiendo -----"

RAR 7.01 x64   Copyright (c) 1993-2024 Alexander Roshal   12 May 2024
Trial version                               Type 'rar -?' for help

Evaluation copy. Please register.

Creating archive .\respaldo12102024211038,80.rar

Adding      .\respaldo12102024211038,80.tar                OK
Done
"----- Enviando a otro directorio -----"
      1 archivo(s) copiado(s).
"----- Descomprimiendo -----"

RAR 7.01 x64   Copyright (c) 1993-2024 Alexander Roshal   12 May 2024
Trial version                               Type 'rar -?' for help

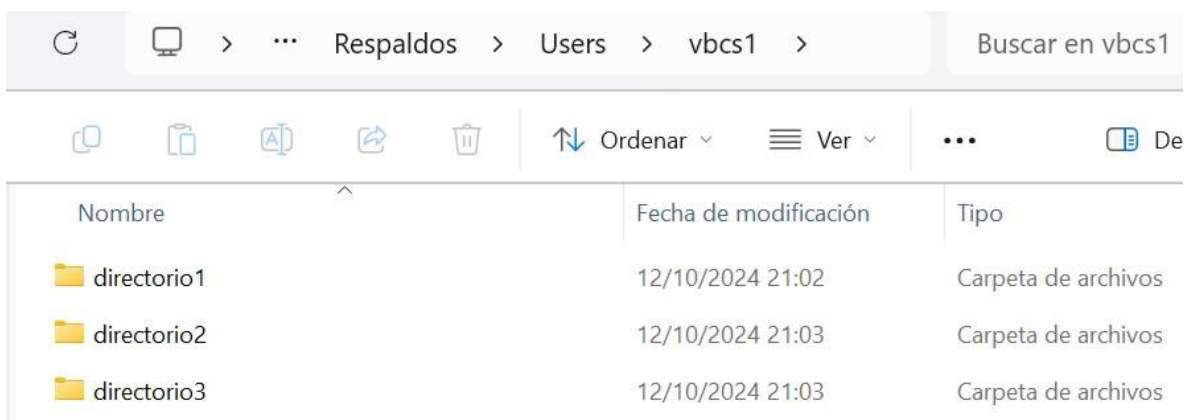
Extracting from C:\Users\vbcs1\Respaldos\respaldo12102024211038,80.rar
```

```
Extracting C:\Users\vbcs1\Respaldos\respaldo12102024211038,80.tar    OK
All OK
"----- Desempacando -----"
x Users/vbcs1/directorio1/
x Users/vbcs1/directorio1/directorio1
x Users/vbcs1/directorio2/
x Users/vbcs1/directorio2/directorio2
x Users/vbcs1/directorio3/
x Users/vbcs1/directorio3/directorio3
"----- Limpiando -----"
"----- Terminado -----"
```

```

C:\Users\vbcs1\Respaldos>dir /s /b
C:\Users\vbcs1\Respaldos\respaldo12102024211038,80.rar
C:\Users\vbcs1\Respaldos\respaldo12102024211038,80.tar
C:\Users\vbcs1\Respaldos\Users
C:\Users\vbcs1\Respaldos\Users\vbcs1
C:\Users\vbcs1\Respaldos\Users\vbcs1\directorio1
C:\Users\vbcs1\Respaldos\Users\vbcs1\directorio2
C:\Users\vbcs1\Respaldos\Users\vbcs1\directorio3
C:\Users\vbcs1\Respaldos\Users\vbcs1\directorio1\directorio1
C:\Users\vbcs1\Respaldos\Users\vbcs1\directorio2\directorio2
C:\Users\vbcs1\Respaldos\Users\vbcs1\directorio3\directorio3

```



Nombre	Fecha de modificación	Tipo
directorio1	12/10/2024 21:02	Carpeta de archivos
directorio2	12/10/2024 21:03	Carpeta de archivos
directorio3	12/10/2024 21:03	Carpeta de archivos

Este script de batch inicia con @echo off para ocultar los comandos y cls para limpiar la pantalla. Luego, captura la fecha y hora actuales, concatenando el día, mes, año, horas, minutos y segundos en la variable fecha. A continuación, crea un archivo tar llamado respaldo%fecha%.tar que incluye los directorios directorio1, directorio2 y directorio3, y lo comprime en un archivo RAR con el mismo nombre. Después, copia el archivo RAR al directorio respaldos y lo descomprime en ese mismo directorio. Posteriormente, desempaca el archivo tar en respaldos y elimina los archivos temporales generados. El script finaliza con un mensaje que indica que ha terminado.

4. Capture y ejecute el siguiente script. Observe su funcionamiento.

```
@echo off
set /p Var1="Introduce valor del primer numero: "
set /p Var2="Introduce valor del segundo numero: "
if %Var1% LSS %Var2% echo %Var1% es menor que %Var2% & goto :seguir_if
echo %Var1% es mayor o igual que %Var2%
if %Var1% EQU 10 echo Fui un 10 & goto :salir
if %Var1% EQU 20 echo Fui un 20 & goto :salir
echo Fui otro valor
goto :salir
:seguir_if
for /L %%V IN (%Var1%,1,%Var2%) do (call :imprime_valor %%V)
goto :salir
:imprime_valor
| echo contador=%1
goto :eof
:salir
| pause
```

```
C:\Users\vbcs1>script3w.bat
Introduce valor del primer numero: 10
Introduce valor del segundo numero: 20
10 es menor que 20
contador=10
contador=11
contador=12
contador=13
contador=14
contador=15
contador=16
contador=17
contador=18
contador=19
contador=20
Presione una tecla para continuar . . .

C:\Users\vbcs1>script3w.bat
Introduce valor del primer numero: 20
Introduce valor del segundo numero: 15
20 es mayor o igual que 15
Fui un 20
Presione una tecla para continuar . . .
```

Este script de batch comienza con @echo off para suprimir la salida de comandos y luego solicita al usuario que introduzca dos números, almacenando estos valores en las variables Var1 y Var2. A continuación, verifica si Var1 es menor que Var2 utilizando la instrucción if. Si es cierto, imprime que Var1 es menor y salta a la etiqueta :seguir_if. Si no, imprime que Var1 es mayor o igual que Var2, y verifica si Var1 es igual a 10 o 20, mostrando mensajes específicos en cada caso y luego saltando a :salir. Si Var1 no es 10 ni 20, imprime "Fui otro valor". Si la condición inicial es verdadera, el script entra en un bucle for que cuenta desde Var1 hasta Var2, llamando a la función :imprime_valor para imprimir cada número en el rango. La función imprime el valor actual del contador. Finalmente, el script llega a la etiqueta :salir, donde se utiliza pause para esperar a que el usuario presione una tecla antes de cerrar la ventana.

5. Capture y ejecute el siguiente script. Observe su funcionamiento.

```
@echo off
echo Opciones:
echo Directorio
echo DirectorioExtendido
echo Salir
set /p Op="Opcion: "
set fin=0
for %%V IN (Directorio DirectorioExtendido Salir) do (call :opcion_elegida %%V, %Op%)
if %fin%==0 echo "Opcion no encontrada"
goto :salir
:opcion_elegida
if "%2"=="%1" if %1==Directorio dir & set fin=1 & goto :EOF else (
if "%2"=="%1" if %1==DirectorioExtendido echo "Similar a dir" & set fin=1 & goto :EOF else (
if "%2"=="%1" if %1==Salir set fin=1 & goto :EOF ))
goto :EOF
:salir
for %%V in (Hola como estan todos) do echo %%V
pause
```

```
C:\Users\vbcs1>script4w.bat
```

```
Opciones:
```

```
Directorio
```

```
DirectorioExtendido
```

```
Salir
```

```
Opcion: 1
```

```
"Opcion no encontrada"
```

```
Hola
```

```
como
```

```
estan
```

```
todos
```

```
Presione una tecla para continuar . . .
```

```
C:\Users\vbcs1>script4w.bat
```

```
Opciones:
```

```
Directorio
```

```
DirectorioExtendido
```

```
Salir
```

```
Opcion: Directorio
```

```
El volumen de la unidad C es Windows-SSD
```

```
El número de serie del volumen es: C6A9-0150
```

```
Directorio de C:\Users\vbcs1
```

12/10/2024	21:19	<DIR>	.
08/06/2024	00:38	<DIR>	..
24/07/2024	19:25		175 .angular-config.json
07/06/2024	20:30	<DIR>	.cache
30/07/2024	20:44	<DIR>	.m2
08/06/2024	17:40	<DIR>	.matplotlib
08/10/2024	12:13	<DIR>	.mcc
08/10/2024	12:13	<DIR>	.mchp_cm
20/09/2024	12:11	<DIR>	.mchp_docs
08/10/2024	12:13	<DIR>	.mchp_packs
20/09/2024	12:11	<DIR>	.mplabcomm
20/09/2024	12:15	<DIR>	.mplab_scripts
19/06/2024	00:52	<DIR>	.nbi
11/10/2024	10:53		176 .packettracer
07/06/2024	20:48	<DIR>	.quokka
12/10/2024	20:20	<DIR>	.VirtualBox
07/06/2024	20:25	<DIR>	.vscode
07/06/2024	20:43	<DIR>	.wallaby
10/10/2024	20:57	<DIR>	antlr

12/10/2024	20:55		343	arit.g4.txt
10/10/2024	21:08		826	arit.interp
10/10/2024	21:08		121	arit.tokens
10/10/2024	21:08		1.900	ArithmeticLexer.interp
10/10/2024	21:08		2.594	ArithmeticLexer.py
10/10/2024	21:08		121	ArithmeticLexer.tokens
10/10/2024	21:08		1.900	aritLexer.interp
10/10/2024	21:08		2.588	aritLexer.py
10/10/2024	21:08		121	aritLexer.tokens
10/10/2024	21:08		1.380	aritListener.py
10/10/2024	21:08		10.048	aritParser.py
09/09/2024	07:59	<DIR>		C
11/10/2024	11:28	<DIR>		Cisco Packet Tracer 8.2.2
07/06/2024	19:08	<DIR>		Contacts
12/10/2024	21:02	<DIR>		directorio1
12/10/2024	21:03	<DIR>		directorio2
12/10/2024	21:03	<DIR>		directorio3
07/06/2024	19:11	<DIR>		Documents
12/10/2024	21:00	<DIR>		Downloads
07/06/2024	19:08	<DIR>		Favorites
20/08/2024	16:48	<DIR>		frontend-tareas
01/07/2024	22:57	<DIR>		GlassFish_Server
07/10/2024	20:20		3.239	ip.c
07/10/2024	20:29		584.869	ip.exe
07/06/2024	19:08	<DIR>		Links
20/09/2024	12:11	<DIR>		MPLABXProjects
07/06/2024	19:08	<DIR>		Music
12/10/2024	20:03	<DIR>		OneDrive
15/09/2024	14:09	<DIR>		Practica2
16/09/2024	17:46	<DIR>		Practica2Copia


```
20/08/2024 16:48 <DIR> frontend-tareas
01/07/2024 22:57 <DIR> GlassFish_Server
07/10/2024 20:20 3.239 ip.c
07/10/2024 20:29 584.869 ip.exe
07/06/2024 19:08 <DIR> Links
20/09/2024 12:11 <DIR> MPLABXProjects
07/06/2024 19:08 <DIR> Music
12/10/2024 20:03 <DIR> OneDrive
15/09/2024 14:09 <DIR> Practica2
16/09/2024 17:46 <DIR> Practica2Copia
10/10/2024 22:11 <DIR> practica4
12/10/2024 20:52 56 prueba
11/08/2024 19:18 <DIR> PycharmProjects
12/10/2024 21:10 <DIR> Respaldos
07/06/2024 19:08 <DIR> Saved Games
12/10/2024 21:09 1.170 script2w.bat
12/10/2024 21:16 505 script3w.bat
12/10/2024 21:19 566 script4w.bat
12/10/2024 20:51 339 scriptw.bat
07/06/2024 21:40 <DIR> Searches
07/09/2024 18:09 <DIR> Ubuntu
12/06/2024 17:10 <DIR> Videos
01/09/2024 19:59 <DIR> VirtualBox VMs
20 archivos 613.037 bytes
42 dirs 346.163.822.592 bytes libres
```

Hola
como
están
todos

Presione una tecla para continuar . . .

```
C:\Users\vbcs1>script4w.bat
Opciones:
Directorio
DirectorioExtendido
Salir
Opcion: DirectorioExtendido
"Similar a dir"
Hola
como
están
todos
Presione una tecla para continuar . . .

C:\Users\vbcs1>script4w.bat
Opciones:
Directorio
DirectorioExtendido
Salir
Opcion: Salir
Hola
como
están
todos
Presione una tecla para continuar . . .
```

Este script de batch comienza ocultando la salida de comandos con @echo off y muestra un menú con las opciones "Directorio", "DirectorioExtendido" y "Salir". Luego, solicita al usuario que introduzca una opción y guarda la respuesta en la variable Op, inicializando fin en 0 para rastrear opciones válidas. Utiliza un bucle for para verificar la opción seleccionada; si es "Directorio", ejecuta dir; si es "DirectorioExtendido", imprime "Similar a dir"; y si es "Salir", simplemente finaliza. Si no se encuentra una opción válida, se muestra un mensaje correspondiente.

Finalmente, imprime las palabras "Hola", "como", "estan" y "todos", y espera que el usuario presione una tecla antes de cerrar.

6. Programe los mismos scripts desarrollados en la sección de Linux ahora desarrollando los correspondientes programas batch en Windows.

Descomprimirw.bat

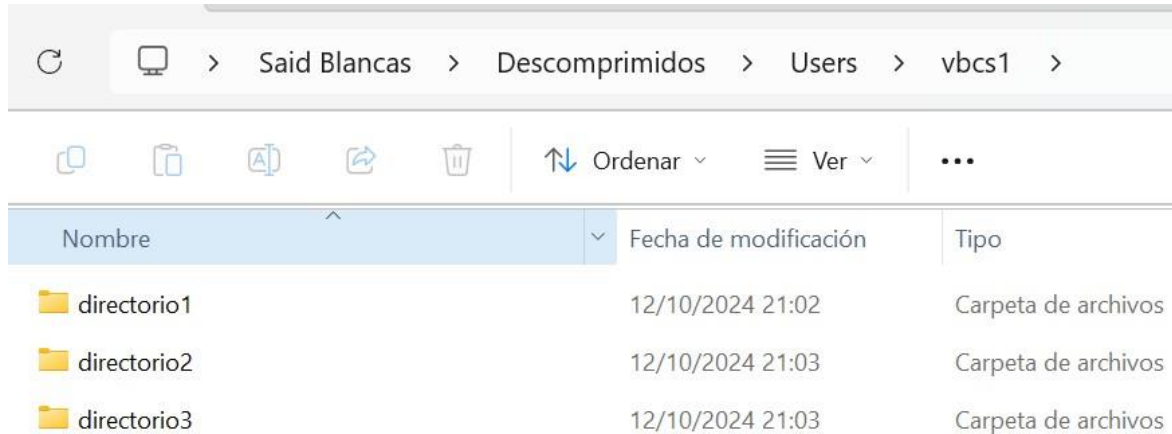
```
@echo off
cls
echo "----- Descomprimiendo Respaldo -----"
setlocal enabledelayedexpansion
set respaldo=C:\Users\vbcs1\Respaldos
set destino=C:\Users\vbcs1\Descomprimidos
if not exist "%destino%" (
    mkdir "%destino%"
)
for %%R in ("%respaldo%\respaldo*.rar") do (
    echo "Descomprimiendo %%R..."
    "C:\Program Files\WinRAR\rar.exe" e -y "%%R" "%destino%"
    set tar=%%~dpnR.tar
    tar -xvf "!tar!" -C "%destino%"
)
echo "----- Descompresión completada -----"
endlocal
```

```
"----- Descomprimiendo Respaldo -----"
"Descomprimiendo C:\Users\vbcs1\Respaldos\respaldo12102024211038,80.rar..."

RAR 7.01 x64   Copyright (c) 1993-2024 Alexander Roshal   12 May 2024
Trial version           Type 'rar -?' for help

Extracting from C:\Users\vbcs1\Respaldos\respaldo12102024211038,80.rar

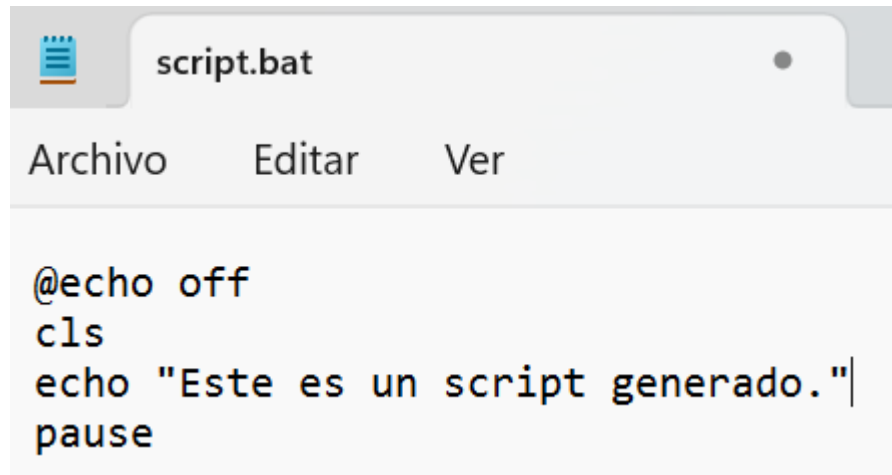
Extracting C:\Users\vbcs1\Descomprimidos\respaldo12102024211038,80.tar OK
All OK
x Users\vbcs1\directorio1/
x Users\vbcs1\directorio1\directorio1
x Users\vbcs1\directorio2/
x Users\vbcs1\directorio2\directorio2
x Users\vbcs1\directorio3/
x Users\vbcs1\directorio3\directorio3
"----- Descompresión completada -----"
```



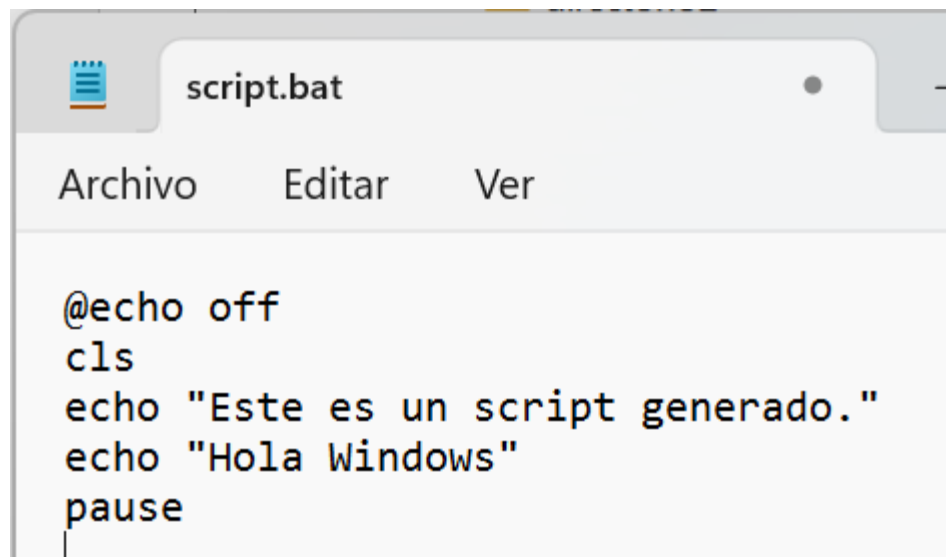
Nombre	Fecha de modificación	Tipo
directorio1	12/10/2024 21:02	Carpeta de archivos
directorio2	12/10/2024 21:03	Carpeta de archivos
directorio3	12/10/2024 21:03	Carpeta de archivos

Generadorw.bat

```
"----- Generador de Script -----"  
"Script 'script.bat' creado."  
"Esperando a que se cierre Notepad para continuar..."
```



```
@echo off  
cls  
echo "Este es un script generado."  
pause
```



```
@echo off  
cls  
echo "Este es un script generado."  
echo "Hola Windows"  
pause
```

```
"Este es un script generado."  
"Hola Windows"  
Presione una tecla para continuar . . .  
"----- Proceso finalizado -----"  
  
C:\Users\vbcs1>
```

Directorioiw.bat

```
@echo off
cls
echo "----- Listado de Archivos -----"
if "%~1"==" " (
    echo "Error: Debe proporcionar un directorio como argumento."
    echo "Uso: %0 <directorio>"
    exit /b
)
set "directorio=%~1"
if not exist "%directorio%" (
    echo "Error: El directorio '%directorio%' no existe."
    exit /b
)
set count=0
echo "Archivos en el directorio '%directorio%':"
for %%F in ("%directorio%\*") do (
    echo %%F
    set /a count+=1
)
echo "-----"
echo "Total de archivos: %count%"
echo "----- Proceso finalizado -----"
```

```
"----- Listado de Archivos -----"
"Archivos en el directorio 'C:\Users\vbcs1\C':"
C:\Users\vbcs1\C\Algoritmo1.c
C:\Users\vbcs1\C\Algoritmo2.c
C:\Users\vbcs1\C\bal.exe
C:\Users\vbcs1\C\balanceo.c
C:\Users\vbcs1\C\bits.c
C:\Users\vbcs1\C\exp.exe
C:\Users\vbcs1\C\expresion.c
C:\Users\vbcs1\C\figura.c
C:\Users\vbcs1\C\hanoi.c
C:\Users\vbcs1\C\hola.c
C:\Users\vbcs1\C\hola.exe
C:\Users\vbcs1\C\holamundo.asm
C:\Users\vbcs1\C\holaMundo.c
C:\Users\vbcs1\C\holaMundo.exe
C:\Users\vbcs1\C\matriz.c
C:\Users\vbcs1\C\par.c
C:\Users\vbcs1\C\pelota.c
"-----"
"Total de archivos: 17"
"----- Proceso finalizado -----"
```

```
C:\Users\vbcs1\C>dir
```

```
El volumen de la unidad C es Windows-SSD
```

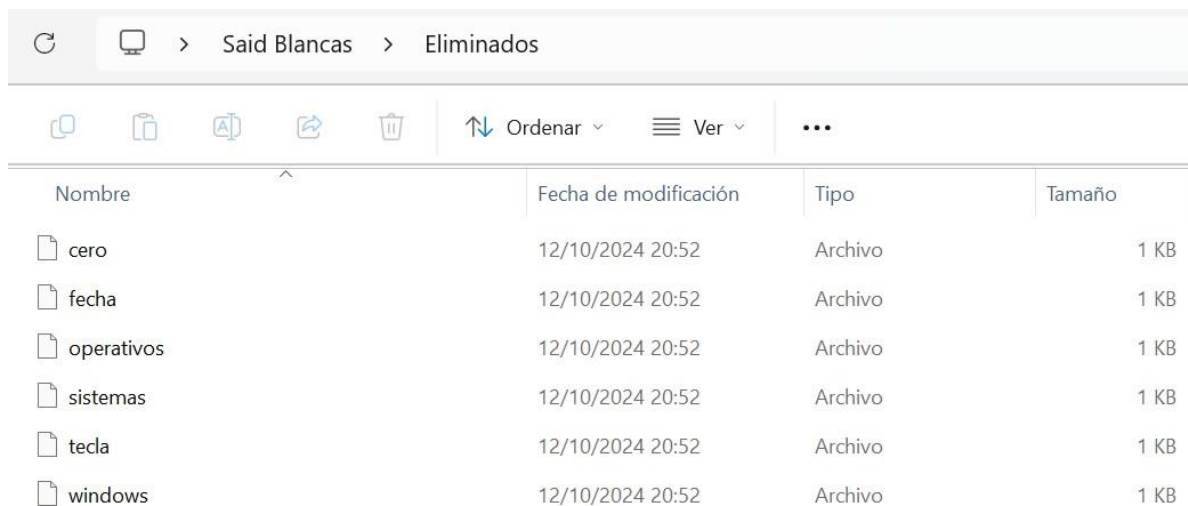
```
El número de serie del volumen es: C6A9-0150
```

```
Directorio de C:\Users\vbcs1\C
```

```
12/10/2024  21:49    <DIR>          .
12/10/2024  21:50    <DIR>          ..
24/08/2024  16:41             212 Algoritmo1.c
24/08/2024  16:47             239 Algoritmo2.c
07/09/2024  18:19        369.034 bal.exe
31/08/2024  14:41         2.309 balanceo.c
04/09/2024  19:18             759 bits.c
07/09/2024  18:22        369.565 exp.exe
07/09/2024  15:41         3.512 expresion.c
09/09/2024  07:59         1.170 figura.c
31/08/2024  14:04             506 hanoi.c
04/09/2024  20:39             124 hola.c
01/09/2024  16:53        367.533 hola.exe
01/09/2024  16:22             708 holamundo.asm
30/08/2024  17:31             77 holaMundo.c
07/09/2024  18:15        367.349 holaMundo.exe
01/09/2024  16:09         1.725 matriz.c
07/09/2024  18:12    <DIR>          output
03/09/2024  20:09             179 par.c
19/06/2024  02:27             137 pelota.c
              17 archivos          1.485.138 bytes
              3 dirs  346.142.064.640 bytes libres
```


Eliminarw.bat

```
@echo off
cls
echo "----- Eliminacion de Archivos -----"
set "carpeta=C:\Users\vbcs1\Eliminados"
if not exist "%carpeta%" (
    echo "Error: El directorio '%carpeta%' no existe."
    exit /b
)
set count=0
setlocal enabledelayedexpansion
echo "Eliminando archivos en el directorio '%carpeta%' con segundo caracter 'e':"
for %%F in ("%carpeta%\*") do (
    set "file=%%~nxF"
    if "!file:~1,1!"=="e" (
        echo "Eliminando: %%F"
        del "%%F"
        set /a count+=1
    )
)
echo "-----"
echo "Total de archivos eliminados: !count!"
echo "----- Proceso finalizado -----"
endlocal
```



Nombre	Fecha de modificación	Tipo	Tamaño
cero	12/10/2024 20:52	Archivo	1 KB
fecha	12/10/2024 20:52	Archivo	1 KB
operativos	12/10/2024 20:52	Archivo	1 KB
sistemas	12/10/2024 20:52	Archivo	1 KB
teda	12/10/2024 20:52	Archivo	1 KB
windows	12/10/2024 20:52	Archivo	1 KB

```
"----- Eliminacion de Archivos -----"
"Eliminando archivos en el directorio 'C:\Users\vbcs1\Eliminados' con segundo caracter 'e':"
"Eliminando: C:\Users\vbcs1\Eliminados\cero"
"Eliminando: C:\Users\vbcs1\Eliminados\fecha"
"Eliminando: C:\Users\vbcs1\Eliminados\teda"
"-----"
"Total de archivos eliminados: 3"
"----- Proceso finalizado -----"
C:\Users\vbcs1>
```

Said Blancas > Eliminados			
Ordenar Ver			
Nombre	Fecha de modificación	Tipo	Tamaño
operativos	12/10/2024 20:52	Archivo	1 KB
sistemas	12/10/2024 20:52	Archivo	1 KB
windows	12/10/2024 20:52	Archivo	1 KB

Conclusión

La práctica de programación en intérpretes de comandos, tanto en Linux como en Windows, ofrece una ventana fundamental al control profundo del sistema operativo y a la automatización de tareas. En un contexto donde la eficiencia y la capacidad para manejar procesos repetitivos son vitales, el scripting en **Bash** y **Batch** se convierte en una herramienta imprescindible para administradores de sistemas, desarrolladores y usuarios avanzados.

Desde una perspectiva teórica y técnica, el scripting a nivel de shell permite transformar lo que podrían ser tareas tediosas y manuales en procesos ágiles y automáticos. Al crear scripts capaces de gestionar archivos, ejecutar comandos del sistema y manipular flujos de control, los usuarios adquieren un control detallado sobre el comportamiento del sistema operativo. Esto no solo mejora la productividad, sino que también reduce la probabilidad de errores humanos, ya que las tareas automatizadas garantizan la consistencia y precisión en la ejecución de procedimientos.

En **Linux**, el intérprete de comandos **Bash** sobresale por su versatilidad, gracias a su rica colección de herramientas nativas y su capacidad para manipular el sistema de archivos, gestionar procesos, y realizar análisis y transformaciones avanzadas de datos. A través de su sistema de variables, funciones y estructuras de control, Bash permite construir scripts robustos que se integran de manera nativa con las diversas utilidades del sistema Unix, lo que resulta en una experiencia de programación eficiente y altamente personalizable. La capacidad de combinar comandos simples para realizar tareas complejas con unas pocas líneas de código es una de las fortalezas distintivas de este entorno.

En **Windows**, aunque el scripting en **Batch** es más limitado en cuanto a flexibilidad y poder en comparación con Bash, sigue siendo una herramienta útil para la automatización de tareas dentro del ecosistema de Windows. Batch

proporciona comandos sencillos que permiten manejar archivos y directorios, lanzar aplicaciones, y realizar operaciones básicas de control de flujo. Si bien la sintaxis y las funcionalidades son más básicas, sigue siendo una herramienta efectiva para gestionar tareas repetitivas en entornos empresariales o personales donde la línea de comandos es adecuada para estas operaciones.

En un nivel más amplio, esta práctica subraya la importancia de la **automatización** en la administración moderna de sistemas. En un mundo donde la eficiencia operativa y la reducción de costos son fundamentales, la capacidad para automatizar tareas cotidianas, como copias de seguridad, gestión de archivos, y la administración de usuarios y permisos, resulta invaluable. Al dominar el scripting en estos entornos, se abre la puerta a la creación de soluciones personalizadas que no solo ahorran tiempo, sino que también optimizan el uso de los recursos del sistema.

Finalmente, la práctica también proporciona una lección sobre la adaptabilidad de los profesionales de TI y desarrolladores. El dominio de ambos entornos —Linux y Windows— permite a los usuarios trabajar de manera transversal en distintas plataformas, adaptando su enfoque a las herramientas disponibles. La capacidad de trasladar conceptos aprendidos en un entorno a otro, como las estructuras de control de flujo, las funciones y la manipulación de archivos, refuerza la versatilidad de quienes se especializan en scripting y automatización.